Open in app ↗

Medium

Access to everything. Now 30% off. Upgrade now

Lumen Engineering… · Follow publication

Distributed Systems + Kubernetes + WebRTC + Websocket + Scalability +

How to implement a distributed and auto-scalable WebSocket server architecture on Kubernetes

17 min read · May 24, 2023



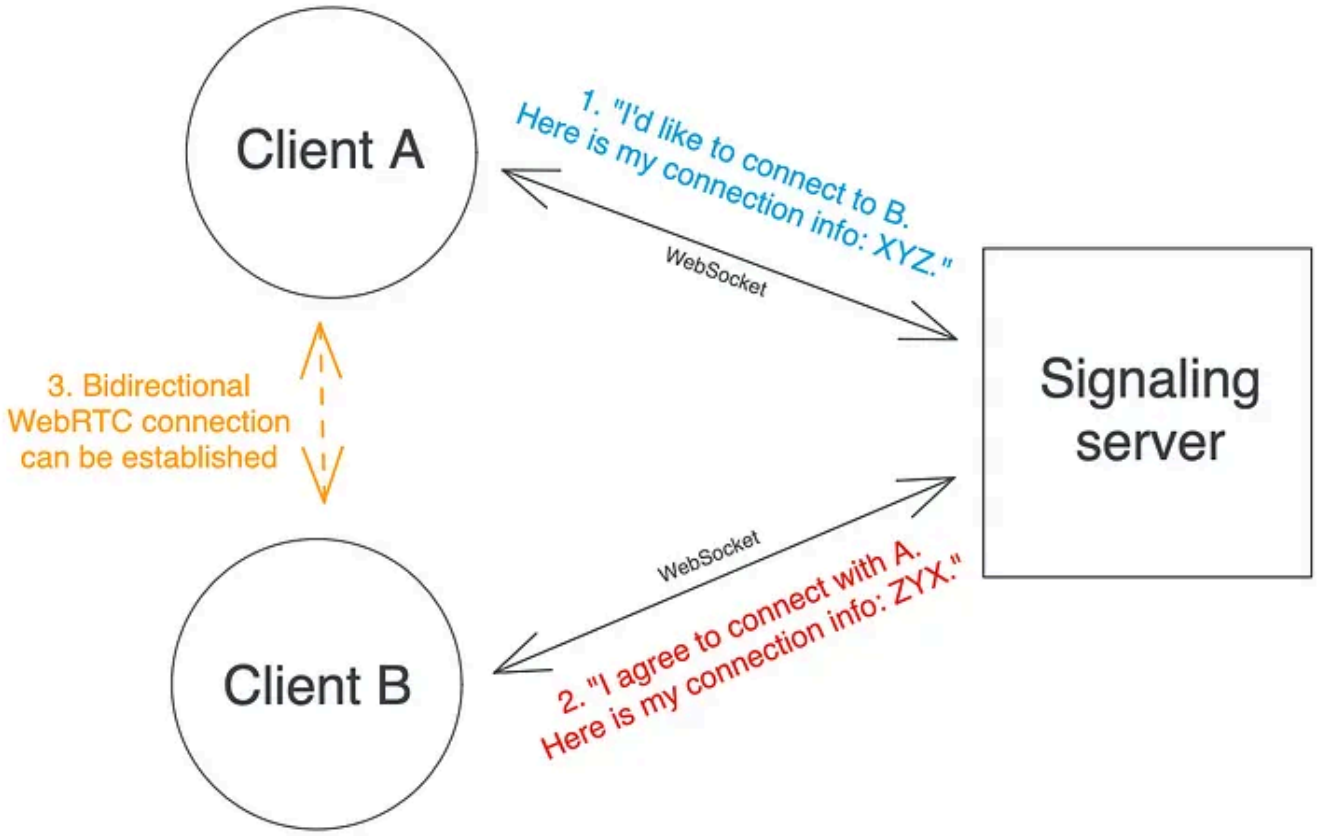
Erwan de Lépinau Follow

Listen Share ••• More

WebRTC signaling: a “live chat”-like use case

As part of our development efforts on the Mesh Delivery technology at Lumen, we needed to design and implement a resilient and scalable backend architecture capable of solving the issue of WebRTC signaling as part of our WebRTC workflow implementation. Do not worry, you do not need to be familiar or have any prerequisites with WebRTC in order to read and learn from this post. Here are the only 2 things you need to know:

- WebRTC is a set of open Web standards that makes it possible to establish direct communication and exchange data between clients (notably web browsers) in a peer-to-peer fashion.
- WebRTC signaling is a preliminary step in the WebRTC protocol that helps with the establishment of a direct connection between clients. It relies on a *signaling server* whose responsibility is to forward negotiation messages between clients who wish to establish a WebRTC connection with each other.



Schematic representation of WebRTC signaling

The signaling server does not need to know or understand the contents of the messages being exchanged (which, if you are curious, usually contain connection information in the SDP format): it just needs to know who the recipient for each message is and forward it to them. The type of connection used between the clients and the signaling server is typically WebSockets because their bidirectional nature is very well suited for this use case. As you can see, this backend use case **is almost identical in all aspects to a live chat service** where users can send arbitrary messages to each other.

Why WebSocket servers are harder to scale than regular HTTP servers

Single instance use case: easy as ABC

In the schema above, there is only one signaling server instance which makes the task relatively straightforward to reason about. Any basic implementation that you would find in "how to create a live chat WebSocket server" online articles should work. It generally boils down to the following:

- Use the language and HTTP library of your choice to start a basic WebSocket server that listens on a given route for requests from clients who wish to *register* to the signaling service. By *register* we mean "open a persistent WebSocket that can later be used to exchange messages with other clients".

- Retain in an in-memory associative structure (e.g. a hashmap/dictionary) a mapping from each registered client's unique identifier — which we will name `clientId` from here on — to their WebSocket.
- Whenever you receive a message (which must, of course, specify the recipient's `clientId`) from a client's WebSocket, look up the recipient in the map of registered clients and forward the message to them via their own WebSocket.

A pseudo-code implementation could look like this:

```
// Global variable that maps each clientId to its associated Websocket.
clientIdToWebsocketMap = new Map()

function main() {
    // Start the server and listen to registration requests.
    server = new WebsocketServer()

    server.listen("/register", registerHandler)
}

function registerHandler(request) {
    // For example, the client can send their clientId as a query parameter.
    clientId = request.queryParams.get("clientId")

    websocket = request.acceptAndConvertToWebsocket()
    websocket.onReceiveMessage = onReceiveMessageHandler

    clientIdToWebsocketMap.insert(clientId, websocket)
}

function onReceiveMessageHandler(message) {
    // We assume the message parsing logic to extract the recipientId from the
    recipientId, body = parseMessage(message)
    clientIdToWebsocketMap.get(recipientId).send(body)
}
```

Autoscaling with multiple instances: asking for trouble

As we just saw a Websocket server architecture with a single server instance is no big deal. However, when you need to handle a **large** and **highly variable** incoming traffic, a single instance will not suffice anymore and would not be an optimal use of your resources: in our real-world use case, we sometimes experience rapid load changes from just a few thousand messages/s to **dozens or even hundreds of**

thousands of messages/s over only a few minutes. This is where features such as Kubernetes' horizontal pod autoscaling (HPA) come in handy: Kubernetes detects the variation in incoming traffic and adds or removes instances as needed.

A first issue when you have several instances is that the sender and recipient for a given message are **not necessarily both registered on the same instance**, in which case you need to find a way to forward the message from the sender's instance to the recipient's instance.

Another issue is with scaling itself. Horizontal autoscaling is designed to play well with regular HTTP services: if traffic increases and the autoscaler determines that a new instance should be created it can simply do so, inform any load balancer (be it a Kubernetes Service, Ingress Controller or any external HTTP load balancer) that sits in front of the instances that a new instance was created, and the load balancer will start distributing new incoming HTTP requests evenly across all instances including the new one.

But here is the catch: Websockets are **persistent connections**. Imagine you have 2 Websocket server instances serving traffic with 100 clients connected to each, and the autoscaler determines that load is high enough for a third instance to be created: it creates that new instance (called a *scale-up* event), informs the load balancer of it, and the load balancer starts distributing new Websocket connection requests evenly across all instances. Can you see the issue? If 30 new Websocket connection requests are distributed across your 3 instances, the older two will have 110 connected clients each whereas the new one will only have 10! **Uneven load is a big no-no** in Kubernetes when using the HPA: the autoscaling expects balanced resource usage across instances.



Why auto-scaling does not play well with persistent connections

Note that this issue is nothing new: load balancing and scaling persistent connections is a well known conundrum in Kubernetes.

Requirements of a scalable distributed signaling service

Before explaining possible solutions and detailing the one we opted for, let's recap the constraints of the problem that we are trying to solve:

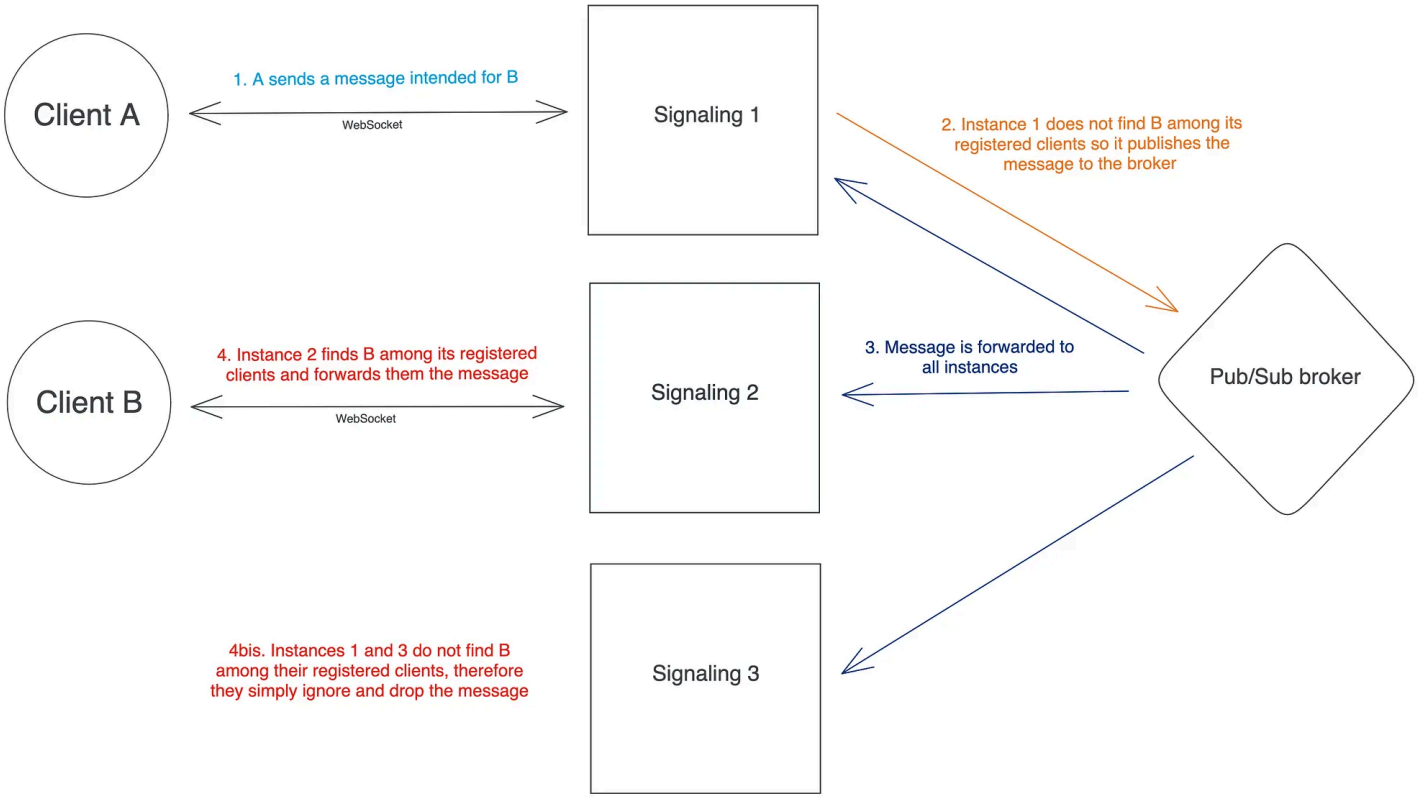
1. *Distributedness (apparently this is an actual word) constraint:* The system must guarantee that a message sent by a client will be correctly forwarded to its intended recipient even if they are **not registered to the same instance** as the sender.

2. *Balance constraint:* In order for the system to be auto-scalable, the load (i.e. the number of active connections) across instances of the system must remain **balanced**, including when an instance is added or removed dynamically from the distributed service by Kubernetes' HPA.

A "canonical" solution that exhibits significant limitations

Using a Pub/Sub broker to solve the Distributedness constraint

If you search for “How to scale Websocket servers” online, you will stumble upon several articles: How to Scale WebSockets on Hackernoon by Hendrik Swanepoel, How to scale WebSocket — horizontal scaling with WebSocket tutorial by Adam Polak or Scaling Websockets by Jo Stichbury just to cite a few of them. A recurring family of solutions that appears under various forms in these articles entails using a **Pub/Sub message broker** to achieve inter-instance communication, which solves the *Distributedness constraint* aka requirement 1 listed above.



Example of a distributed signaling system using a Pub/Sub broker

This proposed solution looks promising on paper as it provides an elegant and decoupled solution to the *Distributedness constraint*. However after further discussion within the team and as we tried to estimate the expected costs of such a solution, we realized it exhibits 2 critical limitations:

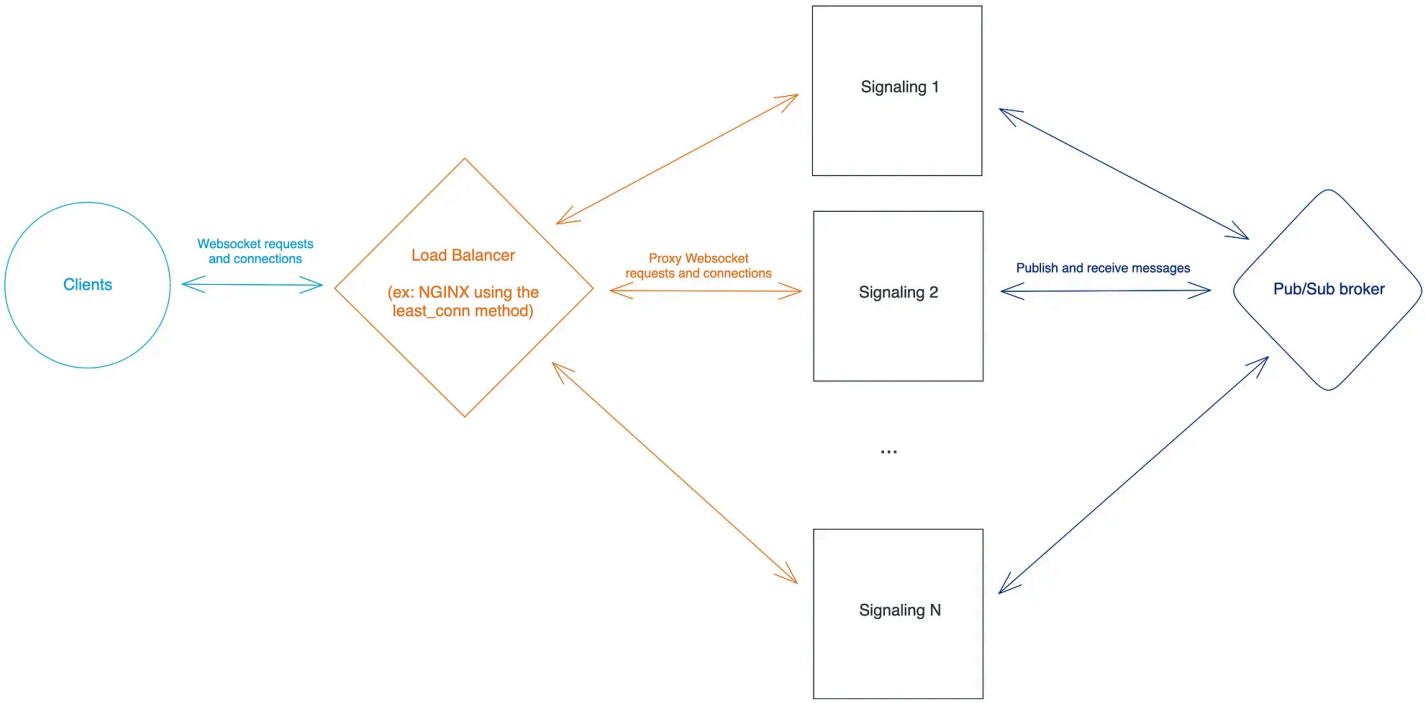
- Each signaling instance is subscribed to the broker and therefore reads every single message published by each of the other instances. This means that the number of reads **scales quadratically with the number of instances.** This is especially inefficient as when an instance reads a message from the broker, there is only on average **a probability of 1/N** (where N is the number of instances) that the recipient is registered to said instance: most of the messages read from the broker will be dropped.

- **It can be very costly**. If your traffic patterns are variable enough — which is probably the case if you needed to implement autoscaling in the first place — you will either need to autoscale the Pub/Sub broker itself or use a managed solution (e.g. Google Pub/Sub if you are using GCP). The former option is a highly complex task (which may even not be doable at all) while the latter can become very expensive very quickly: we realized that our traffic of a few thousand messages transiting each second on average in our signaling system — which is significant but not exceptional by any means— would end up costing us **dozens of thousands of dollars per month** if we were to opt for Google Pub/Sub.

Solving the Balance constraint: the Good and the Ugly

Now for requirement 2, the *Balance constraint*: the article by Jo Stichbury touches upon load balancing methods and mentions a few algorithms that can prove of interest for our use case. The default load balancing algorithms in most load balancers is *round-robin* which, as we explained earlier in this article, works very well for regular HTTP use cases but fails to maintain balanced load when autoscaling Websocket servers. Alternatives such as the *least-connected* algorithm are much more interesting for us: they allow to distribute new Websocket connection requests to the server instance that has the fewest active connections, which guarantees that load will *eventually* be balanced across instances after a scale-up event. *Eventual* balance will suffice in most cases and should not prevent Kubernetes' HPA from working well.

A limitation with this solution to the *Balance constraint* is that the *least-connected* load balancing algorithm is not supported by all load balancers: for instance, Nginx supports it but GCP's HTTP(S) load balancers do not. In that case, you may need to resort to slightly uglier tricks such as leveraging Kubernetes' readiness probes: you can make your most overloaded signaling instances appear temporarily `Unready` to prevent the load balancer from sending them new connection requests. This will redirect all new traffic to the other instances which should help them catch up and even out the load across instances. It's not pretty but it works fine (we are talking from experience here)!



The complete system leveraging a Pub/Sub broker and a load balancer using the least-connected method

To recap, a possible solution for a scalable distributed signaling system would be to use a Pub/Sub broker to achieve inter-instance communication and use a *least-connected* method of load balancing. However, this solution is only viable **IF you can afford the potentially high costs of using a Pub/Sub broker AND your load balancer supports the *least-connected* algorithm.** Since these conditions were not met for us, we decided to design another solution which we are going to present now.

Our solution: leveraging the magic of hash-based load balancing algorithms

Solving the Distributedness constraint using rendezvous hashing

Hash-based load balancing algorithms are deterministic methods of balancing traffic that compute a hash from a value in the client's request (which may be a header value, a query or path parameter, the client's IP address…) and use that hash to route the request to a given backend instance. Two of the best-known hash-based algorithms are consistent hashing and rendezvous hashing. Both are very similar but we ended up opting for rendezvous hashing because it is simpler — both to understand and to implement — and because it tends to balance load more evenly across instances than consistent hashing. However it is worth mentioning that most popular load balancers such as NGINX only provide consistent hashing.



You can see these algorithms as mathematical functions that take as input a value to hash and a set of backend instances, and output one of the backend instances. Running the function with the same values for these arguments will always return the same output value, which is a significant difference compared to most other load-balancing methods (including *round-robin* and *least-connected*).

```
H(val, I) = I_i

- H is the hash-based algorithm
- val is the value (extracted from the request) from which the hash is computed
- I = {I_1, I_2, ..., I_N} is the set of all backend instances
- I_i is the backend instance that was "selected" by the algorithm
```

Can you see where we are going with this? If we use the client's `clientId` as the argument `val`, we now have a load-balancing algorithm that definitively associates each client with a specific signaling instance ; on top of that, anyone who knows their `clientId` and the set of backend instances can run the function themselves at any time to know on which instance that client is. This means that if a signaling instance receives a message intended for a given recipient that it is not found locally, it can run the hash-based algorithm and immediately know on which other instance the recipient is supposed to be registered!

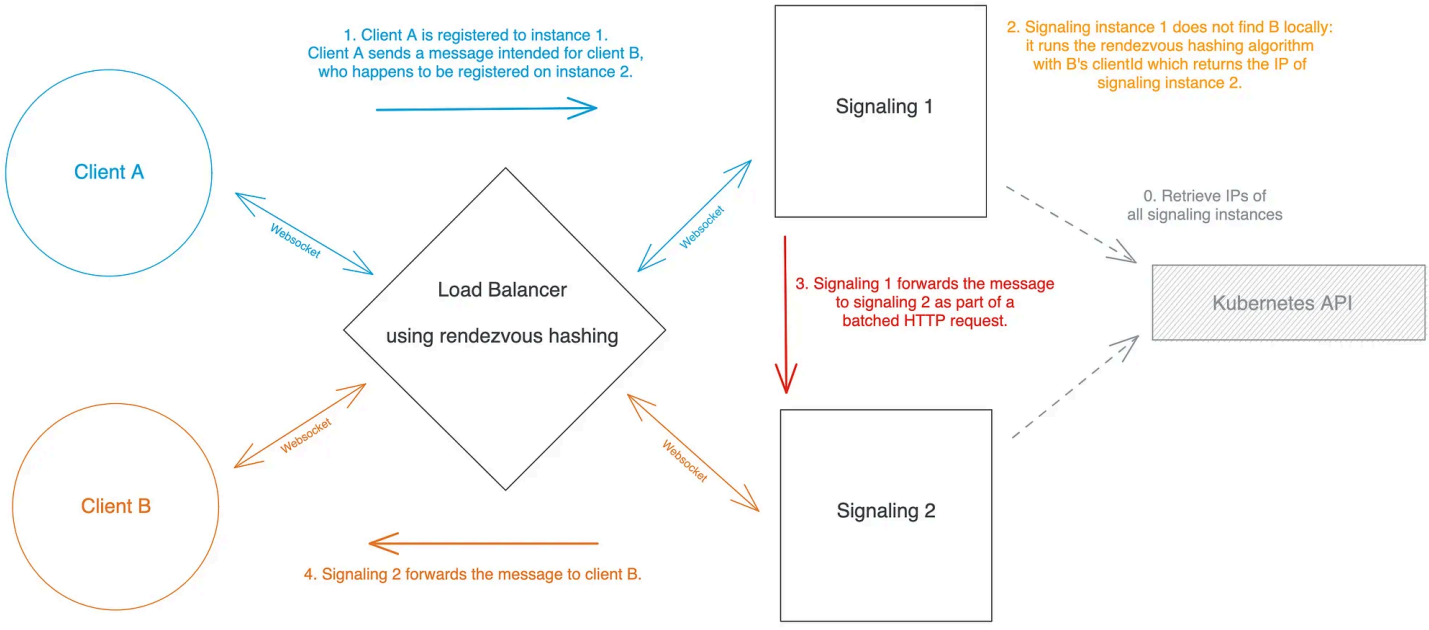
Let's now see how we can put everything together:

- We instruct our load balancer to use rendezvous hashing as its load balancing method, using the `clientId` that is passed in the query parameters of every new Websocket connection request as the value to hash.
- We need to make each signaling instance aware of the other instances in the system: this is easily done in Kubernetes by declaring a Headless Service

associated with our signaling Deployment and then calling the Kubernetes Endpoints API in the signaling server's code to retrieve the set of signaling instances and their IP addresses.

- When a signaling instance I_1 receives a Websocket message from one of its registered clients, it reads the intended recipient's `clientId` and looks for them locally. If they are found locally, the instance forwards the message to them via their Websocket. If they are not found, the instance runs the rendezvous hashing algorithm using the `clientId` as the `val` argument and the set of signaling instance IPs as the `I` argument, and determines on which instance I_2 the recipient is supposed to be registered. If $I_2 = I_1$ it probably means the recipient has disconnected or never even registered in the first place ; otherwise I_1 forwards the message to I_2.

- There are several ways by which I_1 could forward the message to I_2: we chose to use regular HTTP requests on a route specifically created just for inter-instance communication. We chose to send those inter-instance messages as batches in order to avoid sending too many individual HTTP requests (which would cause a lot of performance overhead).



How rendezvous hashing allows for deterministic and efficient inter-instance communication

Tackling the Balance constraint

Using hash-based load balancing algorithms provides an elegant solution to the *Distributedness constraint*, but you may be wondering how it plays with autoscaling which introduces changes (creation or deletion of instances) in the set of signaling

instances while the system is running. A preliminary thing we obviously have to do is make sure we **watch or at least regularly poll** the Kubernetes Endpoints API in the signaling server's code to see when rescales occur: this is an easy enhancement to make from our previous iteration.

A very interesting feature of rendezvous hashing (and consistent hashing) is that when a backend instance is added or removed, which changes the `I` argument to the function, the return value of the function is modified only for a fraction (if the system scales from N-1 to N instances, this fraction is 1/N on average) of the possible input values of `val`. This means that when the signaling service rescales, only **a small portion of the existing registered clients are reassigned** to a different instance — just enough so that load is balanced again across all instances after the rescale.

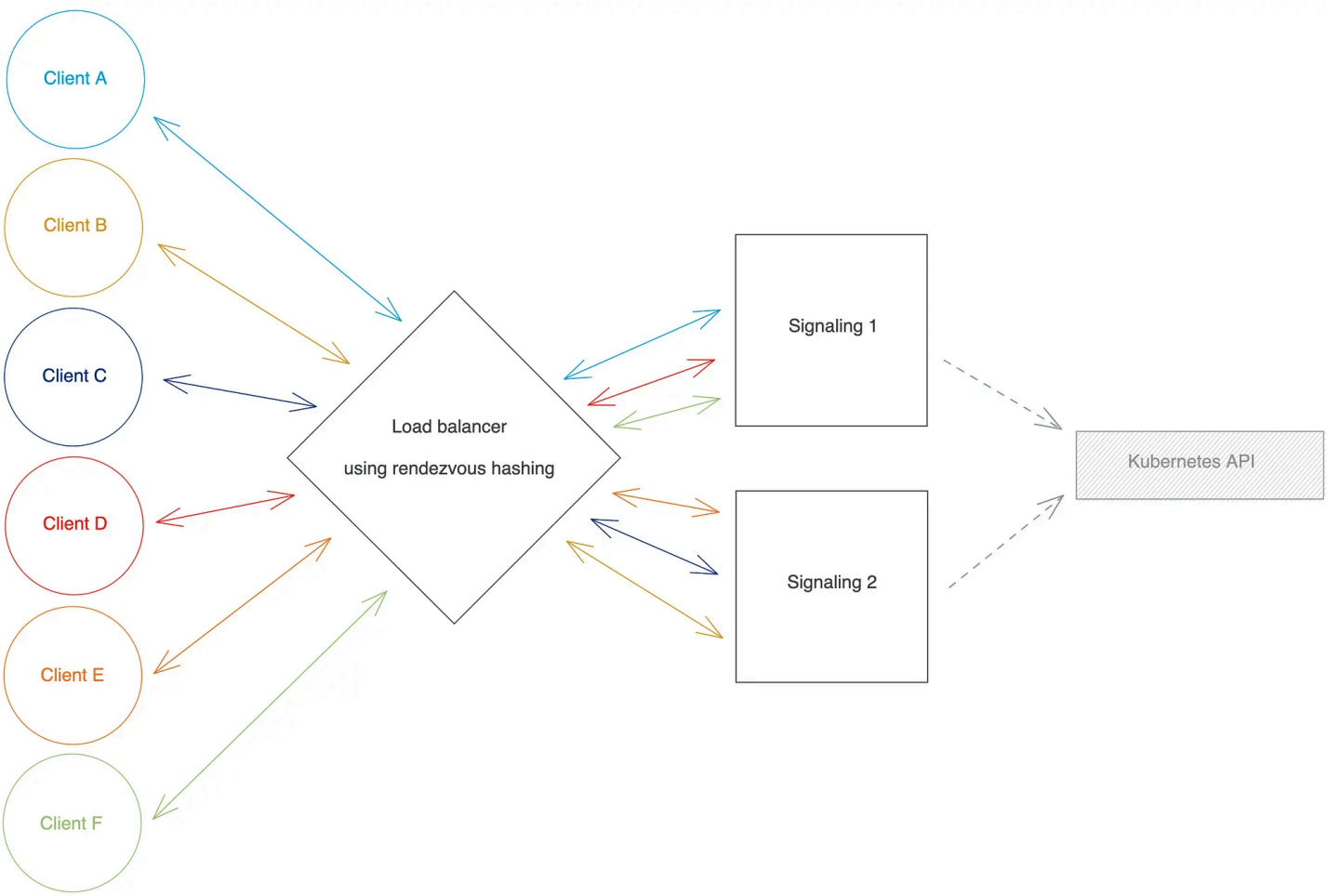
This sounds like great news, but now that we have identified that a few clients should be reassigned a new instance after each rescale, *who is actually going to carry on that reassignment process*? If nobody actually "moves" these Websocket connections from their old instance to the new one, the concerned clients will not receive forwarded messages properly. Indeed, our whole system relies on the fact that each client must be registered to the instance it is assigned to by the rendezvous hashing algorithm! Our initial hope was that load balancers that supported Websockets and hash-based load-balancing methods would be clever enough to do that remapping work for us and… we were wrong. To our knowledge and according to our experiments, **no readily available load balancer (be it Nginx, HAProxy, Envoy…) will do that automatically for you**.

We found two possible solutions to address this last obstacle:

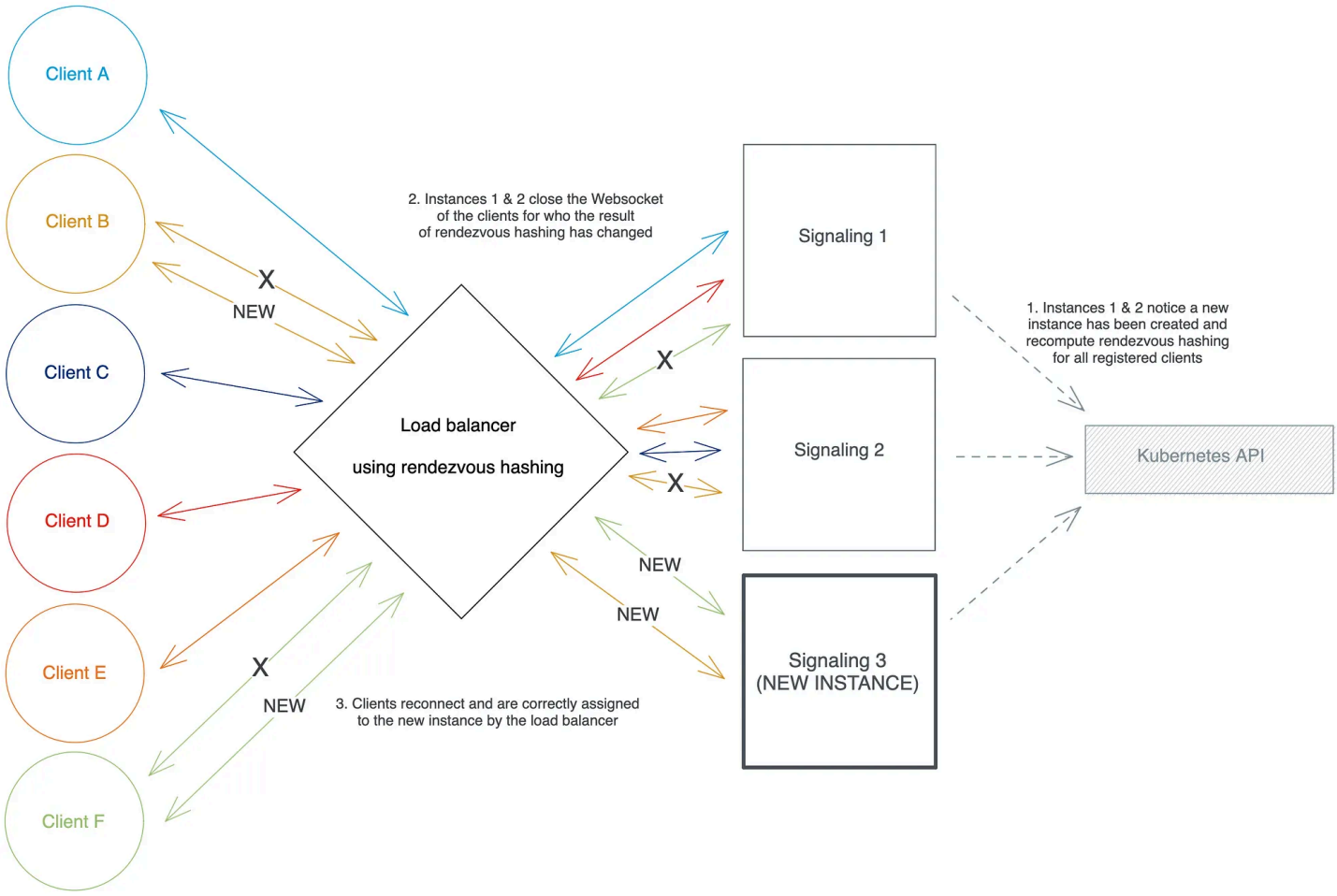
1. Forcing a client reconnection

When a signaling instance I_i detects a rescale event via the Kubernetes Endpoints API, it can go through all the clients registered to it and re-compute the result of the rendezvous hashing algorithm for each `clientId` with the updated set of instances. Logically, a small fraction of these clients should return a new result that is not I_i. The instance I_i can then proceed to **close the Websocket** of these clients: if there is a reconnection mechanism in place in the client code, the clients will send to the signaling system a reconnection request which will go again through the load

balancer —which should now also have the updated set of signaling instances. The load balancer will therefore connect these clients to their appropriate new instance.



Before a rescale event



After a rescale event: solution 1 leveraging client-side reconnection

This solution is conceptually simple, but presents a few drawbacks which may be of importance depending on your use case:

- It requires a reconnection mechanism to be in place in the client code.
- It will temporarily disrupt client sessions by closing their Websocket.
- It further couples your signaling server implementation code to the constraints of the surrounding architecture.
- It will induce reconnection request spikes on your signaling system each time it rescales. This can be mitigated by making the concerned clients reconnect progressively over a given time window instead of all at the same time, at the cost of failing to properly route messages to them until they have reconnected to the correct instance.

For our use case, these drawbacks were too significant so we ended up opting for the following solution which does not rely on any client-side behaviour.

2. Remapping Websockets in the load balancer itself

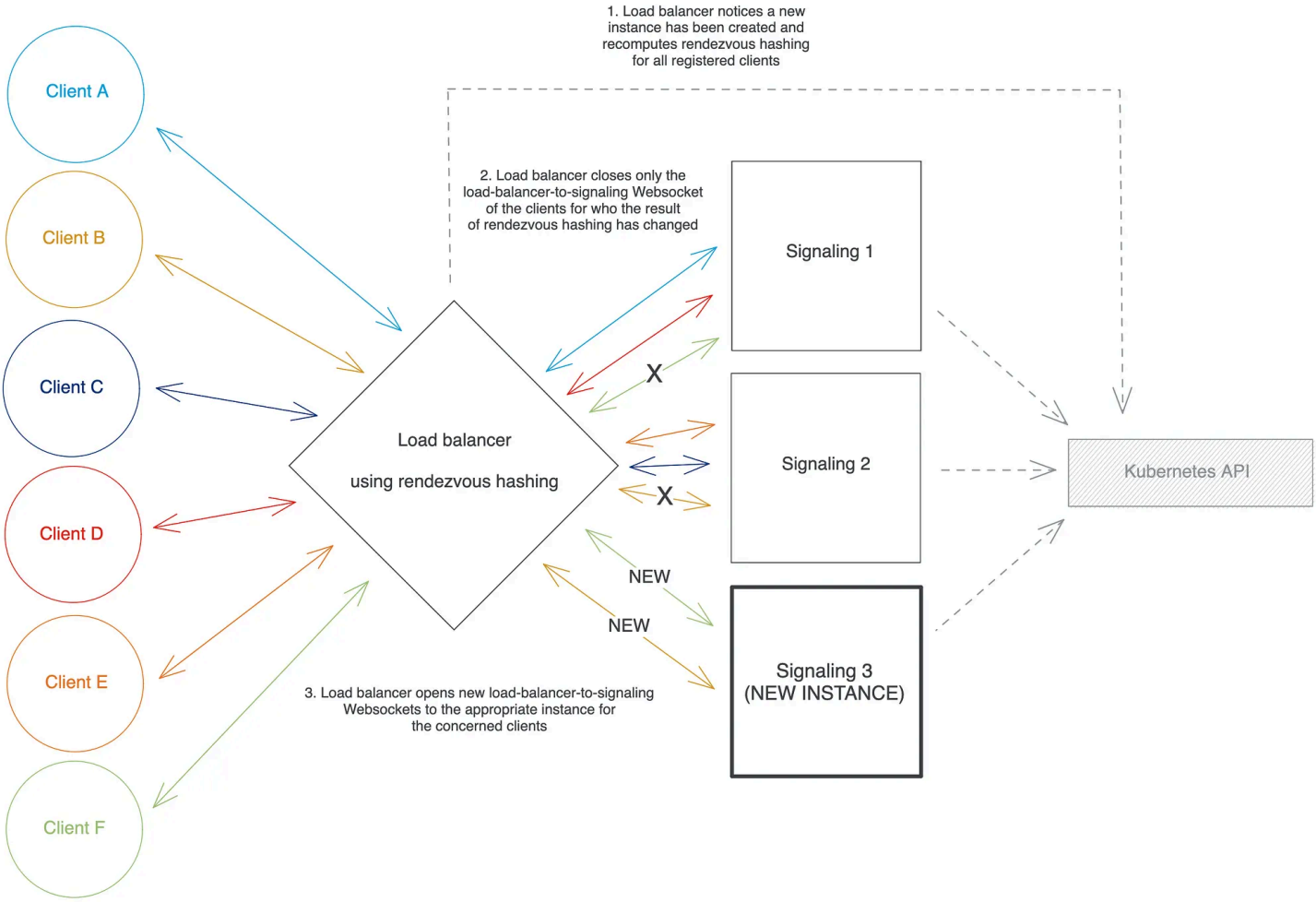
As we said before, ideally we would want the load balancer itself to do the remapping of Websockets when a rescale happens, but no readily available load balancer seems to offer such capability. Therefore we simply decided to… **code our own load balancer**.

Now hold on, do not close the tab just yet! I know you must be thinking *"Are you out of your mind? Load balancers like Nginx or HAProxy are super complex and finely optimized pieces of software that took years and years to develop. You are trying to reinvent the whole car, not just the wheel!"*. But keep in mind that we do not need to implement a fully-featured and versatile load balancer like Nginx or HAProxy here: we only need to be able to proxy Websocket requests and messages, we know exactly what kind of clients and upstream servers we must be compatible with, we know what environment we will be running in (a Kubernetes cluster), we know the specific route we will be listening on and we know what specific load balancing algorithm we will be using. It is also worth mentioning that, in our infrastructure, we have a first layer of load-balancing/reverse proxying in front of our Kubernetes cluster which means that we do not have to handle TLS or ALPN in our custom load balancer.

Under those conditions, coding our own load balancer was actually fairly simple. Here is basically all we needed to implement:

- Signaling instances discovery via Kubernetes API and rendezvous hashing logic: we simply reused the exact same code that we had already implemented in our signaling server in the previous steps.
- Setup a basic Websocket server listening for connection requests on a specific route. The handler for that route runs the rendezvous hashing algorithm with the client's `clietnId`, forwards the request to the selected backend signaling instance, and then forwards the response back to the client. If the response is positive, two Websocket connections are established for that client: one from the client to the load balancer, and another from the load balancer to the signaling instance.
- When the load balancer receives a message on a client-to-load-balancer Websocket, it forwards that message on the load-balancer-to-signaling one for that client, and vice-versa.

Finally, we just needed to implement our Websocket-remapping logic in the event of rescales: when the load balancer notices that the set of signaling instances returned by the Kubernetes API has changed, it goes through all the clients and their `clientId` for which it is currently proxying a Websocket, and re-runs the rendezvous hashing algorithm for each with the newly updated set of backend instances. If this returns a different instance than the one a client is currently registered to, the load balancer **closes only the load-balancer-to-signaling Websocket** for that client (leaving the client-to-load-balancer Websocket untouched) and opens a new load-balancer-to-signaling Websocket to the new signaling instance. It can then resume proxying messages from the client to the signaling system using this new Websocket.



After a rescale event: solution 2 leveraging our custom load balancer

We implemented our Websocket load balancer in Rust using the hyper and tokio-tungstenite libraries: it took us a few weeks to implement it and reach a production-ready state that adheres pretty closely to the Websocket standard with a codebase that amounts to ~2k lines of code including tests, logging, metrics reporting and proper error handling. Our production readings showed that it was “only” short by

10–30% in terms of throughput performance compared to Nginx, without having made any advanced optimizations or used any `unsafe` or highly specialized code.

Now what if you want to autoscale the load balancer itself? You may notice that once again the *Balance constraint* appears — how to maintain an even number of Websockets across load balancer instances as they rescale? — but this time without the *Distributedness constraint* — the task of delivering messages to their recipients is already tackled by the signaling instances. Therefore, the solutions that we previously mentioned for the *Balance constraint*, including using the *least-connected* method in your front proxy/load balancer, should work here. In our case, our front proxy did not allow for *least-connected* load balancing so we had to use the "Readiness probe trick" that we detailed earlier.

Conclusion

We hope you found this article interesting and instructive and would love for it to help you find ideas or solutions to your own problems involving serving Websockets at large scale. On our side, **we have been very satisfied** with our new WebRTC signaling system since officially **deploying it to production in March 2022.** It has served several billions of Websocket connections and enabled clients to exchange close to **a trillion (that is 10^{12}) messages in the last year** without raising a single operational alert or requiring any manual maintenance, which has brought great relief to our on-call team.

I would particularly like to thank Charles-Edouard LATOUR and Thomas MEDIONI who helped me tremendously with this project over the course of almost a year from May 2021 to March 2022. I would also like to thank Reda BENZAIR, Valentin TJONCKE and Igor MUKAM for their precious help, feedback and advice.

As always you can contact me on Twitter (@erwandl) or Linkedin if you have any question or comment. Thank you for taking the time to read this post!

This document is provided for informational purposes only and may require additional research and substantiation by the end user. In addition, the information is provided "as is" without any warranty or condition of any kind, either express or implied. Use of this information is at the end user's own risk. Lumen does not warrant that the information will meet the end user's requirements or that the implementation or usage of this information will result in the desired outcome of the end user. © 2023 Lumen Technologies. All rights reserved.

Distributed Systems + Kubernetes + WebRTC + Websocket + Scalability +

Follow

Published in Lumen Engineering Blog

158 followers · Last published Jun 14, 2023

The Application and Content Delivery arm of Lumen Technologies, we build powerful solutions to deliver, secure and optimize the online experience.



Follow

Written by Erwan de Lépinau

59 followers · 2 following

Software Engineer with a keen interest in anything that’s about zeros and ones